NoSQL (MongoDB)

Professional Study Notes

Complete Reference · Beginner to Advanced

Prepared by

Saran Velmurugan

01. Core Concepts & Architecture

MongoDB is a **document-oriented NoSQL database** that stores data as flexible, JSON-like documents rather than rigid rows and columns. It is schema-free, horizontally scalable, and ideal for modern web applications.

Term	SQL Equivalent	Description
`Database`	Database	A named container that holds collections
`Collection`	Table	A group of related documents (no fixed schema)
`Document`	Row / Record	A single BSON record expressed as JSON
`Field`	Column	A key-value pair within a document
`_id`	Primary Key	Auto-generated unique identifier for every document

Note: *MongoDB stores documents internally as BSON (Binary JSON), which supports more data types than standard JSON, such as Date and ObjectId.*

```
// Sample Document
{
"_id": ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"),
"name": "Saran",
"course": "Full Stack",
"fees": 15000,
"joined": ISODate("2024-01-15")
}
```

02. Shell & Database Commands

Launch the MongoDB interactive shell with:

```
mongosh
```

Database Commands

Command	Description
`use webplusacademy`	Create or switch to a database
`show dbs`	List all databases (only shows DBs with data)
`db`	Show currently selected database
`db.dropDatabase()`	Permanently delete the current database

Collection Commands

Command	Description
`db.createCollection("students")`	Explicitly create a named collection
`show collections`	List all collections in current database
`db.students.drop()`	Delete a collection and all its documents

Tip: *MongoDB auto-creates a collection on first insertOne() — createCollection() is optional but recommended for clarity.*

03. CRUD Operations

3.1 INSERT

insertOne() — add a single document:

```
db.students.insertOne({
name: "Saran",
age: 22,
course: "Full Stack"
})
```

insertMany() — add multiple documents in one call:

```
db.students.insertMany([
{ name: "Arun", age: 21, course: "Java" },
{ name: "Priya", age: 20, course: "Python" },
{ name: "Kavi", age: 23, course: "DevOps" }
])
```

3.2 READ

```
db.students.find() // all documents
db.students.find().pretty() // formatted output
db.students.findOne({ name: "Saran" }) // first matching document
```

Note: *find() returns a cursor. In the shell, it auto-iterates. In code, use .toArray() or iterate manually.*

3.3 UPDATE

$set — modify specific fields without touching others:

```
// Update a single document
db.students.updateOne(
{ name: "Saran" }, // filter
{ $set: { age: 23 } } // update
)

// Update all matching documents
db.students.updateMany(
{ course: "Java" },
{ $set: { course: "Full Stack" } }
)
```

$inc — atomically increment (or decrement) a numeric field:

```
db.students.updateOne(
{ name: "Saran" },
{ $inc: { age: 1 } } // age += 1
)
```

3.4 DELETE

```
db.students.deleteOne({ name: "Arun" }) // first match only
db.students.deleteMany({ course: "Python" }) // all matches
```

Tip: *Always test your filter with find() before running deleteMany() to avoid unintended data loss.*

04. Filtering & Query Operators

MongoDB provides a rich set of comparison and logical operators to query documents with precision.

Operator	Meaning	Example Query
$eq	Equal to	{ age: { $eq: 21 } }
$ne	Not equal to	{ age: { $ne: 21 } }
$gt	Greater than	{ age: { $gt: 20 } }
$gte	Greater than or equal	{ age: { $gte: 21 } }
$lt	Less than	{ age: { $lt: 22 } }
$lte	Less than or equal	{ age: { $lte: 22 } }
$in	Matches any in array	{ course: { $in: ["Java","Python"] } }
$nin	Matches none in array	{ course: { $nin: ["DevOps"] } }

Logical Operators — $and / $or

```
// AND — all conditions must match
db.students.find({
$and: [ { age: { $gte: 20 } }, { course: "Java" } ]
})

// OR — at least one condition must match
db.students.find({
$or: [ { course: "Java" }, { course: "Python" } ]
})

// Implicit AND (shorthand — most common)
db.students.find({ age: 21, course: "Java" })
```

05. Sorting, Limiting & Projection

Sorting

```
db.students.find().sort({ age: 1 }) // ascending (1 = A→Z / low→high)
db.students.find().sort({ age: -1 }) // descending (-1 = Z→A / high→low)
db.students.find().sort({ name: 1, age: -1 }) // multi-field sort
```

Limiting & Skipping

```
db.students.find().limit(5) // first 5 documents
db.students.find().skip(10).limit(5) // pagination: page 3 (if page size = 5)
```

Counting Documents

```
db.students.countDocuments() // total count
db.students.countDocuments({ course: "Java" }) // count with filter
```

Projection — Select Specific Fields

Use a second argument in find() to include (1) or exclude (0) fields:

```
// Include name and course only
db.students.find({}, { name: 1, course: 1, _id: 0 })

// Exclude a field, show everything else
db.students.find({}, { fees: 0 })
```

Note: *You cannot mix include (1) and exclude (0) in the same projection, except for _id which can always be suppressed with _id: 0.*

06. Indexing & Performance

An **index** is a data structure that MongoDB maintains to allow fast lookups. Without an index, MongoDB performs a full *collection scan* — reading every document. With a matching index, it reads only the relevant entries.

```
// Create a single-field index
db.students.createIndex({ name: 1 })

// Compound index (multiple fields)
db.students.createIndex({ course: 1, age: -1 })

// Unique index (no duplicates allowed)
db.students.createIndex({ email: 1 }, { unique: true })

// List all indexes on a collection
db.students.getIndexes()

// Drop a specific index
db.students.dropIndex({ name: 1 })
```

Tip: *Index fields used frequently in find(), sort(), and aggregation $match stages. Avoid over-indexing — every index consumes memory and slows write operations.*

Index Type	Use Case
`Single Field`	Queries filtering/sorting by one field
`Compound`	Queries on multiple fields together
`Unique`	Enforce no duplicate values (e.g., email)
`Text`	Full-text search on string content
`TTL`	Auto-expire documents after a time period

07. Aggregation Pipeline

The **aggregation pipeline** is MongoDB's most powerful feature for data transformation and analysis. Documents pass through a sequence of *stages*, each stage transforming the output of the previous one.

Stage	Purpose
`$match`	Filter documents — equivalent to WHERE in SQL
`$group`	Group documents by a field and compute aggregates
`$sort`	Sort pipeline output
`$limit`	Restrict number of output documents
`$skip`	Skip a number of documents (pagination)
`$project`	Reshape / select fields in output
`$unwind`	Deconstruct an array field into separate documents

Stage	Purpose
`$lookup`	Left outer join with another collection
`$addFields`	Add computed fields to documents
`$count`	Count documents and return as a single field

Example 1 — Count students per course

```
db.students.aggregate([
{ $match: { age: { $gte: 18 } } },
{ $group: { _id: "$course", totalStudents: { $sum: 1 } } },
{ $sort: { totalStudents: -1 } }
])
```

Example 2 — $lookup (JOIN two collections)

Combine data from the **orders** collection with student details:

```
db.orders.aggregate([
{
$lookup: {
from: "students", // collection to join
localField: "student_id", // field in orders
foreignField: "_id", // field in students
as: "studentInfo" // output array field name
}
},
{ $unwind: "$studentInfo" } // flatten the array
])
```

Accumulator Operators (used inside $group)

Accumulator	Description
`$sum`	Sum of numeric values (use 1 to count)
`$avg`	Average of numeric values
`$min`	Minimum value in the group
`$max`	Maximum value in the group
`$push`	Collect values into an array
`$first`	First document value in the group

08. User Authentication & Security

MongoDB supports role-based access control (RBAC). Always enable authentication in production environments.

```
// Switch to admin database
use admin

// Create an administrator
db.createUser({
user: "adminUser",
pwd: "StrongP@ssw0rd!",
roles: [{ role: "root", db: "admin" }]
```

```
})

// Create a read-only user for a specific database
use webplusacademy
db.createUser({
user: "readonlyUser",
pwd: "AnotherStr0ng!",
roles: [{ role: "read", db: "webplusacademy" }]
})
```

Role	Permissions
`read`	Read-only access to a specific database
`readWrite`	Read and write access to a specific database
`dbAdmin`	Administrative tasks — indexes, stats, compaction
`userAdmin`	Create and manage users on a database
`root`	Superuser — full access to all databases (use with caution)

Note: *Never assign the 'root' role to application service accounts. Use the minimum privilege role required.*

09. Practice Lab — Student Management System

Complete all tasks below in sequence using the MongoDB shell. This exercise reinforces every concept covered in these notes.

01	**Insert**	Add 10 student documents with fields: name, age, course, fees, joinedDate
02	**Read**	Retrieve all students using find() and examine the auto-generated _id field
03	**Filter**	Find all students where age > 20 using the $gt operator
04	**Compound**	Find students in 'Java' course AND age >= 21 using $and
05	**Update**	Update the course of one student using updateOne() with $set
06	**Bulk Update**	Change all 'Python' students to 'Data Science' using updateMany()
07	**Delete**	Remove one student by name using deleteOne()
08	**Sort**	List all students sorted by age descending, limit to top 5
09	**Projection**	Display only name and course — hide _id using projection
10	**Count**	Count total students, then count per course using countDocuments()
11	**Index**	Create an index on the course field. Check with getIndexes()
12	**Aggregate**	Use aggregate with $group to count the number of students per course

Notes compiled by **Saran Velmurugan**

NoSQL (MongoDB) — Complete Professional Reference